

IHK Algorithmen Handbuch 2025 – PRO

Ausführliche Erklärungen · Java-ähnlicher Pseudocode · **IHK-Stil-Aufgaben mit Lösung & Schritt-für-Schritt** · Tipps & Fehlerfallen

1) Zählen / Summieren / Durchschnitt

sehr häufig

Prinzip: Ein Array einmal durchlaufen, Bedingung prüfen, Zähler/Akkumulator anpassen.
Durchschnitt = Summe / Anzahl.

Pseudocode

```
funktion zaehleGroesser(a, grenze)
    count = 0
    for (i = 0; i < a.length; i++)
        if (a[i] > grenze) count++
    return count
```

```
funktion durchschnitt(a)
    sum = 0
    for (i = 0; i < a.length; i++)
        sum = sum + a[i]
    return sum / a.length
```

IHK-Aufgabe (vereinfacht)

Aufgabe: Zähle, an wie vielen Tagen die Temperatur > 25°C war. a = {22, 27, 29, 23, 26}.

Lösung: 27, 29, 26 → 3

Schritte: count=0 → 22 nein → 27 ja → 29 ja → 23 nein → 26 ja → return 3.

Tipps: Zähler mit 0 starten; Rückgabe erst *nach* der Schleife.

2) Min / Max (1D)

sehr häufig

Prinzip: Mit erstem Element starten, dann vergleichen und ggf. übernehmen. Für *Index* zusätzlich minIndex/maxIndex pflegen.

Pseudocode

```
funktion findeMin(a)
  min = a[0]
  for (i = 1; i < a.length; i++)
    if (a[i] < min) min = a[i]
  return min
```

```
funktion findeMax(a)
  max = a[0]
  for (i = 1; i < a.length; i++)
    if (a[i] > max) max = a[i]
  return max
```

IHK-Aufgabe (vereinfacht)

Aufgabe: Finde den kleinsten Messwert in $a = \{5, 7, 2, 9, 3\}$.

Lösung: min=5 → 7 nein → 2 ja (min=2) → 9 nein → 3 nein → **2**.

Fehler: Startwert 0 verwenden; return in der Schleife platzieren.

3) Min / Max (2D) + Objektrückgabe

sehr häufig

Prinzip: Für jede Zeile (Monat) Summe bilden; daraus min/max der Monatssummen ermitteln. Ergebnis als Objekt zurückgeben.

Pseudocode

```
funktion ermittleJahresstatistik(verbrauch) // Monate × Tage
  min = MAX_WERT; max = MIN_WERT
  for (m = 0; m < verbrauch.length; m++)
    summe = 0
```

```

    for (t = 0; t < verbrauch[m].length; t++)
        summe = summe + verbrauch[m][t]
    if (summe < min) min = summe
    if (summe > max) max = summe
    return new Jahresstatistik(min, max)

```

```

klasse Jahresstatistik(min, max)
// Attribute: minVerbrauch, maxVerbrauch
// Konstruktor & Getter

```

IHK-Aufgabe (Sommer 2022 – stilisiert)

Aufgabe: Gegeben ist ein 2D-Array *verbrauch[12][tage]*. Bestimme den minimalen und maximalen *Monatsverbrauch* und gib ein Objekt *Jahresstatistik* zurück.

Schritt-für-Schritt:

1. *min* = MAX, *max* = MIN initialisieren.
2. Jeden Monat *m*: *summe*=0 → alle Tage addieren.
3. Nach dem Monat: mit *min/max* vergleichen und ggf. setzen.
4. Am Ende: `return new Jahresstatistik(min, max)`.

Typische Falle: Tageswerte direkt vergleichen (falsch) statt Monatssumme zu bilden.

4) If/Else + Counter (Aktie besser als DAX)

häufig

Prinzip: Pro Tag die prozentuale Veränderung Aktie vs. DAX vergleichen; wenn Aktie > DAX → Zähler erhöhen.

Pseudocode

```

funktion notierungPlus(kurse)
    count = 0
    for (i = 0; i < kurse.length; i++)
        if (kurse[i].aktieProzent > kurse[i].daxProzent) count++
    return count

```

IHK-Aufgabe (Sommer 2022 – stilisiert)

Datenstruktur Tageskurs hat *aktieProzent*, *daxProzent*. Bestimme, an wie vielen Tagen die Aktie „besser“ war.

Schritte: count=0 → Schleife → wenn Bedingung wahr, count++ → am Ende return count.

Fallen: $\geq$ statt $>$; Rückgabe in der Schleife.

5) Lineare Suche häufig

Pseudocode

```
funktion linearSearch(a, x)
  for (i = 0; i < a.length; i++)
    if (a[i] == x) return i
  return -1
```

IHK-Aufgabe (vereinfacht)

Suche Artikelnummer 4711 in $a=\{4700, 4711, 4800\}$ → Rückgabe **1**, sonst **-1**.

Hinweis: Bei mehreren Treffern reicht oft der erste Index; sonst zählen.

6) Binäre Suche (sortiert) seltener

Pseudocode

```
funktion binarySearch(a, x)
  left = 0; right = a.length - 1
  while (left <= right)
    mid = (left + right) / 2
    if (a[mid] == x) return mid
    if (a[mid] < x) left = mid + 1
```

```
    else right = mid - 1
return -1
```

IHK-Aufgabe (vereinfacht)

$a=\{2,4,7,9,11\}$, $x=7 \rightarrow \text{Mitte}=7 \rightarrow \text{Index } 2$.

Falle: Array muss sortiert sein; sonst falsches Ergebnis.

7) BubbleSort möglich

Pseudocode

```
funktion bubbleSort(a)
  for (i = 0; i < a.length - 1; i++)
    for (j = 0; j < a.length - 1 - i; j++)
      if (a[j] > a[j+1]) swap(a[j], a[j+1])
```

IHK-Aufgabe (vereinfacht)

$a=\{5,2,4\}$: $i=0 \rightarrow (5,2) \text{ swap} \rightarrow \{2,5,4\}$; $(5,4) \text{ swap} \rightarrow \{2,4,5\}$; $i=1 \rightarrow \text{ok} \rightarrow \text{Ergebnis } \{2,4,5\}$.

Falle: Innere Grenze $n-1-i$ falsch.

8) InsertionSort möglich

Pseudocode

```
funktion insertionSort(a)
  for (i = 1; i < a.length; i++)
    key = a[i]
    j = i - 1
    while (j >= 0 && a[j] > key)
      a[j+1] = a[j]
```

```
j = j - 1  
a[j+1] = key
```

IHK-Aufgabe (vereinfacht)

a={5,2,4}: i=1 key=2 → schiebe 5 → setze 2; i=2 key=4 → schiebe 5 → setze 4 → {2,4,5}.

Falle: a[j+1]=key nach der while nicht vergessen.

9) Objektvergleich mit vergleiche(x,y)

oft bei Objekten

Pseudocode

```
funktion sort(a, vergleiche)  
  for (i = 0; i < a.length - 1; i++)  
    for (j = 0; j < a.length - 1 - i; j++)  
      if (vergleiche(a[j], a[j+1]) > 0) swap(a[j], a[j+1])
```

IHK-Aufgabe (vereinfacht)

Aufgabe: Sortiere Tageskurse nach Datum. vergleiche(x,y) liefert >0, wenn x nach y.

Hinweis: Nie direkt x > y bei Objekten – immer die Vergleichsfunktion verwenden.

10) Strings & Prüfziffern (EAN/ISBN-13, Luhn)

sehr prüfungsnah

EAN/ISBN-13 (1/3 · Mod-10)

```
funktion pruefzifferEAN13(z12) // 12 Ziffern  
  sum = 0  
  for (i = 0; i < 12; i++)  
    z = z12[i]  
    w = (i % 2 == 0) ? 1 : 3  
    sum = sum + z * w
```

```
p = (10 - (sum % 10)) % 10
return p
```

Luhn (Kreditkarte)

```
funktion luhnGueutig(s) // nur Ziffern
  sum = 0; parity = s.length % 2
  for (i = 0; i < s.length; i++)
    d = s[i]
    if (i % 2 == parity)
      d = d * 2
      if (d > 9) d = d - 9
    sum = sum + d
  return (sum % 10) == 0
```

IHK-Aufgabe (vereinfacht): Berechne die Prüfziffer für „400638133393“ (12 Stellen). Zeige Rechenschritte. Ergebnis $p = (10 - (S \bmod 10)) \bmod 10$.

Prüfungstipps (goldene Regeln)

- Immer Variablen initialisieren (count=0, sum=0, min=a[0]).
- Äußere/innere Schleifen korrekt setzen, besonders bei 2D.
- Rückgaben *nach* der Schleife; in der Schleife nur bei Suche.
- Bei Objekten: Ergebnis als `new Klassenname(...)` zurückgeben.
- Vergleichsfunktion bei Objekten statt direkter Operatoren.

Druck: Browser → Drucken → Als PDF speichern (weißes Layout).